# ARCHITECTURE

# LLMOrchestrator Architecture

## Overview

Standalone Go module (`digital.vasic.llmorchestrator`) for managing headless CLI coding agents with hybrid pipe+file communication. Orchestrates multiple agent types (Claude Code, OpenCode, Gemini, Junie, Qwen Code) through a unified interface with pooling, health monitoring, and circuit breaking.

## Package Structure

```
pkg/
  agent/     -- Agent interface, AgentPool, MultiProviderPool,
HealthMonitor, CircuitBreaker
  adapter/   -- BaseAdapter + 5 CLI agent adapters
  protocol/  -- PipeTransport (JSON-lines) and FileTransport
(inbox/outbox/shared)
  parser/    -- ResponseParser (JSON/action/issue extraction from
raw LLM output)
  config/    -- .env loading, agent path resolution, validation
cmd/
  orchestrator/ -- CLI entry point
```

## Agent Management

### Agent Interface

```go
type Agent interface {
    ID() string
    Name() string
    Start(ctx context.Context) error
    Stop(ctx context.Context) error
```

```
    IsRunning() bool
    Health(ctx context.Context) HealthStatus
    Send(ctx context.Context, prompt string) (Response, error)
    SendStream(ctx context.Context, prompt string) (<-chan
        StreamChunk, error)
    SendWithAttachments(ctx context.Context, prompt string,
        attachments []Attachment) (Response, error)
    OutputDir() string
    Capabilities() AgentCapabilities
    SupportsVision() bool
    ModelInfo() ModelInfo
}
```

### Agent Pool

`AgentPool` is thread-safe (`sync.Mutex` + `sync.Cond`):

- `Register(agent)` -- adds an agent to the pool
- `Acquire(ctx, requirements)` -- blocks until a matching agent is available (supports context cancellation)
- `Release(agent)` -- returns an agent for reuse
- `HealthCheck(ctx)` -- runs health checks on all registered agents
- `Shutdown(ctx)` -- gracefully stops all agents

### MultiProviderPool

Manages separate pools per provider type. `AgentSelector` interface chooses the best provider for a given request. Default: round-robin selection.

Supported providers: `opencode`, `claude-code`, `gemini`, `junie`, `qwen-code`.

## Adapter Layer

`BaseAdapter` provides shared process management (start/stop/health). Each concrete adapter only implements parsing:

| Adapter | Agent | Communication |
|---|---|---|
| `OpenCodeAdapter` | OpenCode CLI | Pipe (stdin/stdout) |
| `OpenCodeHeadlessAdapter` | OpenCode headless | Pipe (JSON-lines) |
| `ClaudeCodeAdapter` | Claude Code CLI | Pipe (stdin/stdout) |
| `GeminiAdapter` | Gemini CLI | Pipe (stdin/stdout) |
| `JunieAdapter` | JetBrains Junie | File (inbox/outbox) |
| `QwenCodeAdapter` | Qwen Code CLI | Pipe (stdin/stdout) |

# Communication Protocols

## PipeTransport

JSON-lines over stdin/stdout. Each message is a single JSON object terminated by newline. Used by most adapters.

## FileTransport

Inbox/outbox/shared directory-based exchange. The orchestrator writes a prompt file to the agent's inbox; the agent writes its response to the outbox. Used by Junie and other agents that lack pipe support.

# Response Parser

`ResponseParser` extracts structured data from raw LLM text output:

- JSON block extraction (fenced code blocks)
- Action extraction (file edits, commands, tool calls)
- Issue extraction (errors, warnings)
- Security: path traversal protection, response length limits, API key masking

# Fault Tolerance

- **CircuitBreaker**: 3 consecutive failures opens the circuit for 60 seconds. Prevents cascading failures when an agent process is unresponsive.
- **HealthMonitor**: Periodic health checks on all agents. Tracks status, latency, consecutive failures.
- **Graceful shutdown**: Pool shutdown stops all agents in order, respecting context deadlines.

# Key Design Decisions

- **BaseAdapter pattern**: Shared process lifecycle logic reduces duplication across 5+ adapters.
- **Blocking Acquire**: `sync.Cond` enables efficient waiting without polling. Context cancellation ensures no goroutine leaks.
- **Protocol abstraction**: Pipe vs. file transport is hidden behind the Agent interface. Callers do not know or care how the agent communicates.
- **Security-first parsing**: All extracted file paths are validated against traversal attacks. Response sizes are bounded.